JAVA PROGRAMMING

Chapter 23: Exception Handling in Java

Personal Notes

Real-world code is full of things that can go wrong — files that don't exist, network calls that fail, division by zero, null values where there shouldn't be any. Without a plan, even one of these makes your entire program crash.

Exception handling is Java's way of dealing with these problems gracefully. Instead of crashing, your code can catch the problem, recover, log it, or fail in a controlled way. It's one of the features that separates beginner code from production-ready code.

1. What is an Exception?

An exception is an UNEXPECTED EVENT that happens during program execution and disrupts the normal flow. When an exception occurs, Java creates an EXCEPTION OBJECT and "throws" it. Unless you catch it, the program crashes.

Common Real-World Examples

- Dividing a number by zero
- Accessing an array index that doesn't exist
- Calling a method on a null reference
- Trying to open a file that doesn't exist
- Parsing "abc" as an integer
- Network connection dropping mid-request

Without Exception Handling — Program Crashes

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        int c = a / b;          // ArithmeticException – / by zero!
        System.out.println("This line never runs");
    }
}

// Output:
// Exception in thread "main" java.lang.ArithmeticException: / by zero
//     at Main.main(Main.java:5)
// Process terminated.
```

With Exception Handling — Program Continues

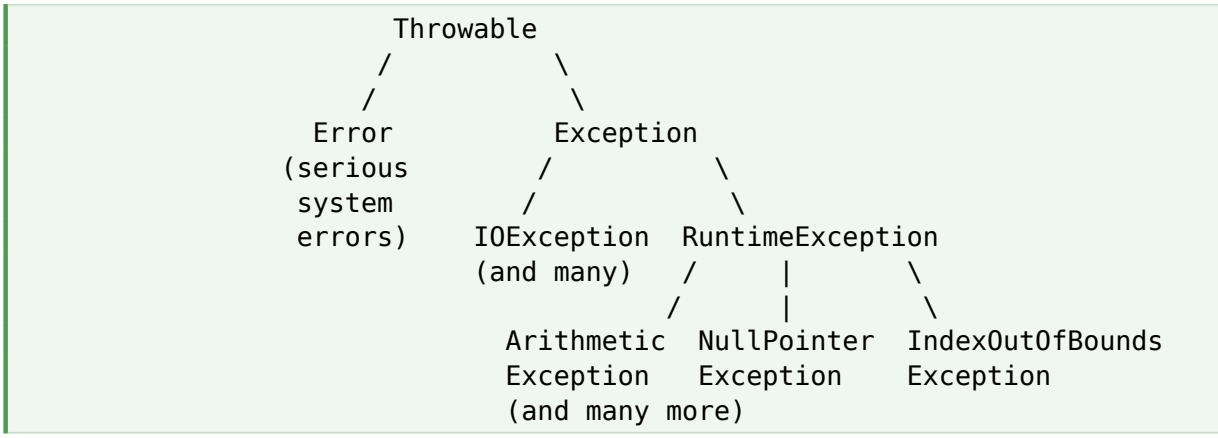
```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int c = a / b;      // exception happens here
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        System.out.println("Program continues normally.");
    }
}

// Output:
// Caught: / by zero
// Program continues normally.
```

2. The Exception Hierarchy

Every exception in Java is an object. They all live in a class hierarchy under Throwable.



2.1 Two Big Categories

Type	Meaning	Examples
Error	Serious problems — JVM crashes, hardware issues, out of memory	OutOfMemoryError, StackOverflowError
Exception	Things that can be handled in code	IOException, ArithmeticException

In your code, you'll deal almost exclusively with Exception (and its subclasses). Errors are usually outside your control — you don't catch OutOfMemoryError.

3. Checked vs Unchecked Exceptions

Inside the Exception class, Java divides exceptions into two types based on whether the compiler FORCES you to handle them.

3.1 Checked Exceptions

The compiler REQUIRES you to handle these (either with try-catch or by declaring throws). If you don't, your code won't compile.

- Examples: IOException, SQLException, FileNotFoundException, ClassNotFoundException
- They represent problems that can reasonably be expected — like "a file might not exist"
- Force the programmer to think about recovery

```
// This won't compile — IOException is a checked exception
public void readFile() {
    FileReader fr = new FileReader("data.txt");   // COMPILE ERROR
}

// Fix 1: handle it
public void readFile() {
    try {
        FileReader fr = new FileReader("data.txt");
    } catch (IOException e) {
        e.printStackTrace();
    }
}

// Fix 2: declare it
public void readFile() throws IOException {
    FileReader fr = new FileReader("data.txt");
}
```

3.2 Unchecked Exceptions (RuntimeException)

The compiler does NOT force you to handle these. They usually represent programming bugs — things you should fix, not catch.

- Examples: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, NumberFormatException, ClassCastException
- Usually caused by bad logic or invalid input
- You CAN catch them, but the better fix is often to prevent them

```
// Compiles fine — but crashes at runtime
String s = null;
int len = s.length();        // NullPointerException

int[] arr = {1, 2, 3};
```

```
int x = arr[10];              // ArrayIndexOutOfBoundsException

int n = Integer.parseInt("abc");   // NumberFormatException
```

Quick Comparison

Aspect	Checked	Unchecked
Compiler check	YES — must handle or declare	NO — compiler ignores
When to use	Recoverable, expected problems	Bugs / invalid input
Examples	IOException, SQLException	NPE, ArrayIndexOOB, ArithmeticException
Parent class	Exception (directly)	RuntimeException (subclass of Exception)
Common approach	Catch and recover	Fix the underlying bug

4. The try-catch Block

The try-catch block is the heart of exception handling. You put risky code inside try, and Java jumps to catch if something goes wrong.

4.1 Basic Syntax

```
try {
    // risky code that might throw an exception
} catch (ExceptionType e) {
    // what to do if the exception happens
}
```

4.2 Simple Example

```
public class Main {
    public static void main(String[] args) {
        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[10]);    // throws exception
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of range!");
        }

        System.out.println("Program continues...");
    }
}
```

```
// Output:
// Index out of range!
// Program continues...
```

4.3 How It Works

```
            try {
                line 1
                line 2  ← exception thrown HERE
                line 3  ← skipped
                line 4  ← skipped
            }
            catch (...) {
                this block runs immediately
            }
            continue here ←  normal flow resumes
```

5. Multiple Catch Blocks

A single try block can have MULTIPLE catch blocks — one for each exception type. Java checks them in order and runs the FIRST one that matches.

```
try {
    String s = null;
    int x = s.length();              // NullPointerException
    int[] arr = new int[5];
    arr[10] = 1;                     // would be ArrayIndexOutOfBounds
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array problem");
} catch (NullPointerException e) {
    System.out.println("Null problem");
} catch (Exception e) {
    System.out.println("Some other problem");   // catches anything else
}
```

> **Important rule:** Order your catch blocks from MOST SPECIFIC to MOST GENERAL. Java picks the first matching catch — so if you put catch (Exception e) first, none of the others will ever run. The compiler will actually stop you from doing this.

5.1 Multi-Catch (Java 7+)

If multiple exceptions need the same handling, you can combine them with the pipe (|) operator.

```
try {
    riskyCode();
} catch (IOException | SQLException | ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
    log(e);
```

```
}
```

6. The finally Block

The finally block runs ALWAYS — whether an exception happened or not, whether it was caught or not, even if a return statement was hit. Use it for cleanup code that MUST run.

```
try {
    System.out.println("Try block");
    int x = 10 / 0;          // exception!
} catch (ArithmeticException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("Finally always runs");
}

// Output:
// Try block
// Caught: / by zero
// Finally always runs
```

6.1 When finally Runs

Scenario	Finally runs?
No exception happens	**Yes — finally runs**
Exception happens and is caught	**Yes — finally runs**
Exception happens but is NOT caught	**Yes — finally still runs, then exception propagates**
return in try block	**Yes — finally runs before the return**
System.exit() in try block	**NO — System.exit() bypasses finally**

6.2 What finally is For

finally is typically used to clean up resources — close files, close database connections, release locks. Anything that absolutely MUST happen, regardless of what went wrong.

```
FileReader fr = null;
try {
    fr = new FileReader("data.txt");
    // use the file
} catch (IOException e) {
    System.out.println("Error reading file");
} finally {
```

```
    if (fr != null) {
        try {
            fr.close();        // ALWAYS close, even on error
        } catch (IOException e) {
            // ignore close errors
        }
    }
}
```

7. try-with-resources (Java 7+)

The cleanup code above is ugly. Java 7 added try-with-resources to handle it automatically. Any resource declared in the try() parentheses gets closed for you — no finally needed.

Before — Old Style

```
FileReader fr = null;
try {
    fr = new FileReader("data.txt");
    // use file
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (fr != null) try { fr.close(); } catch (IOException e) {}
}
```

After — try-with-resources

```
try (FileReader fr = new FileReader("data.txt")) {
    // use file
    // fr.close() is called AUTOMATICALLY when the block ends
} catch (IOException e) {
    e.printStackTrace();
}
```

Rule for try-with-resources: The resource in parentheses must implement AutoCloseable (almost all I/O classes do). Java guarantees close() will be called, even if an exception is thrown. This is the modern, recommended way to handle files, streams, database connections, etc.

Multiple Resources

```
try (
    FileReader fr = new FileReader("input.txt");
    FileWriter fw = new FileWriter("output.txt")
) {
    // use both — both will be closed automatically
}
```

8. The throw Keyword

You can manually throw an exception using the throw keyword. This is useful when you detect an invalid state and want to signal the caller.

```
public class BankAccount {
    private double balance;

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
        if (amount > balance) {
            throw new RuntimeException("Insufficient balance");
        }
        balance -= amount;
    }
}

// Usage:
BankAccount acc = new BankAccount();
acc.withdraw(-100);   // throws IllegalArgumentException
```

Notice the keyword is throw (singular), not throws. Common mix-up.

9. The throws Keyword

throws (with an s) is used in a METHOD DECLARATION to say "this method might throw these exceptions — callers, beware." It's how you propagate a checked exception upward instead of handling it yourself.

```
public void readFile(String name) throws IOException {
    FileReader fr = new FileReader(name);   // might throw IOException
    // ... use file
}

public void caller() {
    try {
        readFile("data.txt");
    } catch (IOException e) {
        System.out.println("Couldn't read file");
    }
}
```

throw vs throws

Aspect	throw	throws
Keyword	throw	throws

Aspect	throw	throws
Where used	Inside a method body	In the method signature
What it does	Actually throws an exception object	Declares which exceptions might be thrown
Syntax	throw new IllegalArgumentException(...)	void foo() throws IOException, SQLException
Followed by	An exception object (new ...())	Exception class names

10. Custom Exceptions

Sometimes the built-in exceptions don't quite match your situation. You can create your OWN exception class to represent application-specific errors with meaningful names.

10.1 Creating a Custom Exception

```
// Custom checked exception
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom unchecked exception
public class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Extend Exception for a checked exception (caller must handle it). Extend RuntimeException for an unchecked one (optional to handle).

10.2 Using a Custom Exception

```
public class BankAccount {
    private double balance = 1000;

    public void withdraw(double amount) throws
InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(
                "Not enough funds. Available: " + balance);
        }
        balance -= amount;
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        try {
            acc.withdraw(5000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

When to use custom exceptions: Use them when your application has domain-specific errors that built-in types don't capture well. "InsufficientBalanceException" reads way better than catching a generic RuntimeException with a vague message.

11. Common Built-in Exceptions

Exception	When it happens
`NullPointerException`	Calling a method on a null reference: null.length()
`ArrayIndexOutOfBoundsException`	Accessing an array with an invalid index: arr[100]
`StringIndexOutOfBoundsException`	Accessing a String with an invalid index: s.charAt(100)
`ArithmeticException`	Math errors like divide by zero: 10/0
`NumberFormatException`	Parsing a String that isn't a number: Integer.parseInt("abc")
`ClassCastException`	Invalid cast: (String) someInteger
`IllegalArgumentException`	A method got an argument it can't accept
`IllegalStateException`	Method called when object is in wrong state
`IOException`	I/O failure: file not found, can't read, etc. (checked)
`FileNotFoundException`	Trying to open a file that doesn't exist (checked)
`SQLException`	Database query failure (checked)
`ConcurrentModificationException`	Modifying a collection while iterating it

12. Useful Exception Methods

Every exception object inherits these useful methods from Throwable:

- getMessage() — returns the message you passed to the exception
- toString() — returns the exception's class name + message
- printStackTrace() — prints the full stack trace to standard error
- getCause() — returns the underlying cause if one was set
- getStackTrace() — returns the stack trace as an array

```
try {
    int x = Integer.parseInt("abc");
} catch (NumberFormatException e) {
    System.out.println(e.getMessage());       // For input string: "abc"
    System.out.println(e.toString());         //
java.lang.NumberFormatException: ...
    e.printStackTrace();                       // full stack trace
}
```

13. Common Mistakes

Mistake 1: Catching too broadly

```
// BAD — hides all problems
try {
    riskyCode();
} catch (Exception e) {
    // do nothing
}
```

Catching Exception (or worse, Throwable) and ignoring it hides bugs and makes debugging impossible. Catch specific types and handle them meaningfully.

Mistake 2: Empty catch blocks

```
try {
    doSomething();
} catch (IOException e) {
    // silently ignored — terrible!
}
```

Even if you can't handle it, at least log it. Silent failures are nightmares to debug.

Mistake 3: Confusing throw and throws

throw (no s) ACTUALLY throws an exception. throws (with s) DECLARES that a method might throw one. Easy typo, but they do completely different things.

Mistake 4: Catching exceptions instead of fixing the bug

```
// BAD — covers up a bug
try {
    String s = getName();
    int len = s.length();
} catch (NullPointerException e) {
    // we just hide that getName() returns null
}

// GOOD — handle the null up front
String s = getName();
if (s != null) {
    int len = s.length();
}
```

Mistake 5: Using exceptions for normal control flow

Exceptions are EXPENSIVE — they involve building a stack trace. Don't use them as glorified if-statements. They're for EXCEPTIONAL situations, not regular logic.

Mistake 6: Catching general before specific

```
try {
    riskyCode();
} catch (Exception e) {                 // matches everything!
    ...
} catch (IOException e) {                // NEVER reached → compile error
    ...
}
```

14. Best Practices

- Catch specific exceptions, not the generic Exception class
- Never use empty catch blocks — at least log the error
- Use try-with-resources for any AutoCloseable resource (files, streams, sockets)
- Throw exceptions early (validate inputs at method entry), catch them late
- Use custom exceptions for domain-specific errors with meaningful names
- Always include a useful error message when throwing
- Don't use exceptions for normal control flow — they're expensive
- Order catch blocks from most specific to most general
- Prefer prevention (null checks, validation) over catching and recovering

15. Quick Reference

Keyword / Concept	Meaning
`try { }`	Wrap code that might throw an exception
`catch (Type e) { }`	Handle a specific exception type
`finally { }`	Cleanup code that always runs
`throw new ...()`	Manually throw an exception
`throws Type, ...`	Declare exceptions a method might throw
`try (Resource r)`	Auto-close resources (try-with-resources)
`Checked`	Compiler requires handling (IOException, etc.)
`Unchecked`	Compiler doesn't force handling (NPE, etc.)
`Throwable`	Root of all exceptions and errors
`Exception`	Recoverable problems
`RuntimeException`	Unchecked — usually programming bugs
`e.getMessage()`	The message you passed when throwing
`e.printStackTrace()`	Print the full stack trace

16. Practice Problems

Try these to internalize exception handling.

1. Write code that takes two numbers and divides them. Handle division by zero with a try-catch.
2. Take user input as a String. Try to parse it as an integer. Handle NumberFormatException with a friendly message.
3. Create an array of size 5. Try to access index 10. Catch the exception and print a useful error.
4. Write a method getAge() that throws IllegalArgumentException if age is negative or above 150.
5. Create a custom InsufficientBalanceException. Use it in a BankAccount withdraw() method.
6. Write a method that demonstrates try, catch, AND finally — show that finally runs in both success and failure cases.
7. Use multi-catch to handle ArithmeticException and NumberFormatException with the same logic.
8. Read a file using try-with-resources. Compare it to the same code using try-finally.

9. Create a custom checked exception InvalidEmailException. Throw it from a validateEmail(String) method.
10. Demonstrate the difference between throw and throws with one example of each.
11. Build a calculator class. Throw appropriate exceptions for divide-by-zero, negative square root, etc.
12. Write code that intentionally triggers 5 different exceptions and catches each with a specific message.

17. Final Takeaway

Exception handling is what turns brittle code into resilient code. Beginners avoid exceptions and hope nothing goes wrong. Real developers design WITH exceptions in mind — knowing what can fail, deciding what to recover from, and writing code that stays running even when individual operations fail.

The goal is not to wrap everything in try-catch "just in case." It's to be intentional: catch what you can actually handle, let the rest bubble up, prevent what you can with validation, and always leave the system in a clean state.

> **Key Takeaway:** Use try-catch for things outside your control (files, networks, user input). Prevent NullPointerExceptions and ArrayIndexOutOfBounds through validation, not catching. Use try-with-resources for any closeable resource. Always include meaningful messages. The rule is simple: catch what you can fix, prevent what you can foresee.

18. What's Next?

Now that exceptions are clear, the natural next Java topics are:

- Lambda Expressions & Functional Interfaces (Java 8+)
- Method References — the :: operator
- Java Streams API — functional operations on collections
- Optional<T> — modern alternative to handling null
- File I/O — reading and writing files with Files, Path, BufferedReader
- Multithreading — Thread, Runnable, synchronized, concurrent collections
- Date and Time API (java.time)